In Class Activity 3 — CRC Cards and Class Diagram

CIS 641 Systems Analysis and Design · Artie Bowman

The term project is PersonaSync™, a tool that keeps a user's declared context (who they are, how they like to work, what they are working on) in one place and syncs that context into the AI tools they use. These cards and the diagram are a rough first pass at the structural model for that system.

1. CRC Cards

1.1 Persona

A Persona is one bundle of context a user keeps. It is the main thing the system stores and moves around.

> *Example: a "Grad School" persona that holds my program, my writing voice, and the courses I am in this term.*

Class: Persona	
Responsibilities	**Collaborators**
• Know its name, description, and tone • Hold an ordered set of ContextBlocks • Add, remove, and reorder blocks • Render itself into a requested format • Know whether it is the active persona	• User • ContextBlock • SyncAdapter

1.2 SyncAdapter

The SyncAdapter does the translating and pushing. It takes a persona and a target tool and produces the text that tool wants, then sends it. Keeping this in its own class means adding a new AI tool later is one new adapter, not a change to Persona.

> *Example: push the "Grad School" persona to Claude as a project instruction and to ChatGPT as custom instructions, each within that tool's character limit.*

Class: SyncAdapter	
Responsibilities	**Collaborators**
• Translate a Persona into one AITool's prompt format • Respect the tool's character limit and trim by block priority • Push the rendered text to the tool and pull it back • Track last sync time and status	• Persona • AITool • ClientApp

1.3 ToolPreference

ToolPreference is the optional decision memory of which persona a user pairs with which tool. The user reviews and keeps these pairings, and the menu-bar app uses them to suggest the right persona when a tool is opened.

> *Example: I usually use the "Grad School" persona in Claude, so the app offers it first when Claude is in front.*

Class: ToolPreference	
Responsibilities	**Collaborators**
• Record each time a Persona is used with an AITool • Keep a use count and last-used time • Suggest the most likely Persona for a tool	• Persona • AITool • ClientApp

2. Rough Class Diagram

Eight classes. Here is what each one is and how it connects:

- **User** owns the Personas, DataSources, and ClientApps.
- **Persona** is made of ContextBlocks (composition). It is the thing that travels between tools.
- **ContextBlock** is one piece of context inside a Persona, with a category, priority, and source.
- **AITool** is a target like Claude or ChatGPT, with its prompt format and character limit.
- **SyncAdapter** links one Persona to one AITool and does the translate and push, so each of those lines reads 1 to 0..*.
- **ToolPreference** also links one Persona to one AITool and remembers which pairing the user kept. The ClientApp asks it for a suggestion.
- **DataSource (MCP)** is an input like Wispr Flow or a calendar that feeds new ContextBlocks.
- **ClientApp** is the web app, the macOS menu-bar app, or the browser extension. It uses the SyncAdapter to do the work.

Still rough: ClientApp will likely split into WebApp, MenuBarApp, and BrowserExtension subclasses, and the opt-in usage-sharing pipeline is left off until it is a real requirement.

See attached diagram, Figure 1 ↓

PersonaSync™

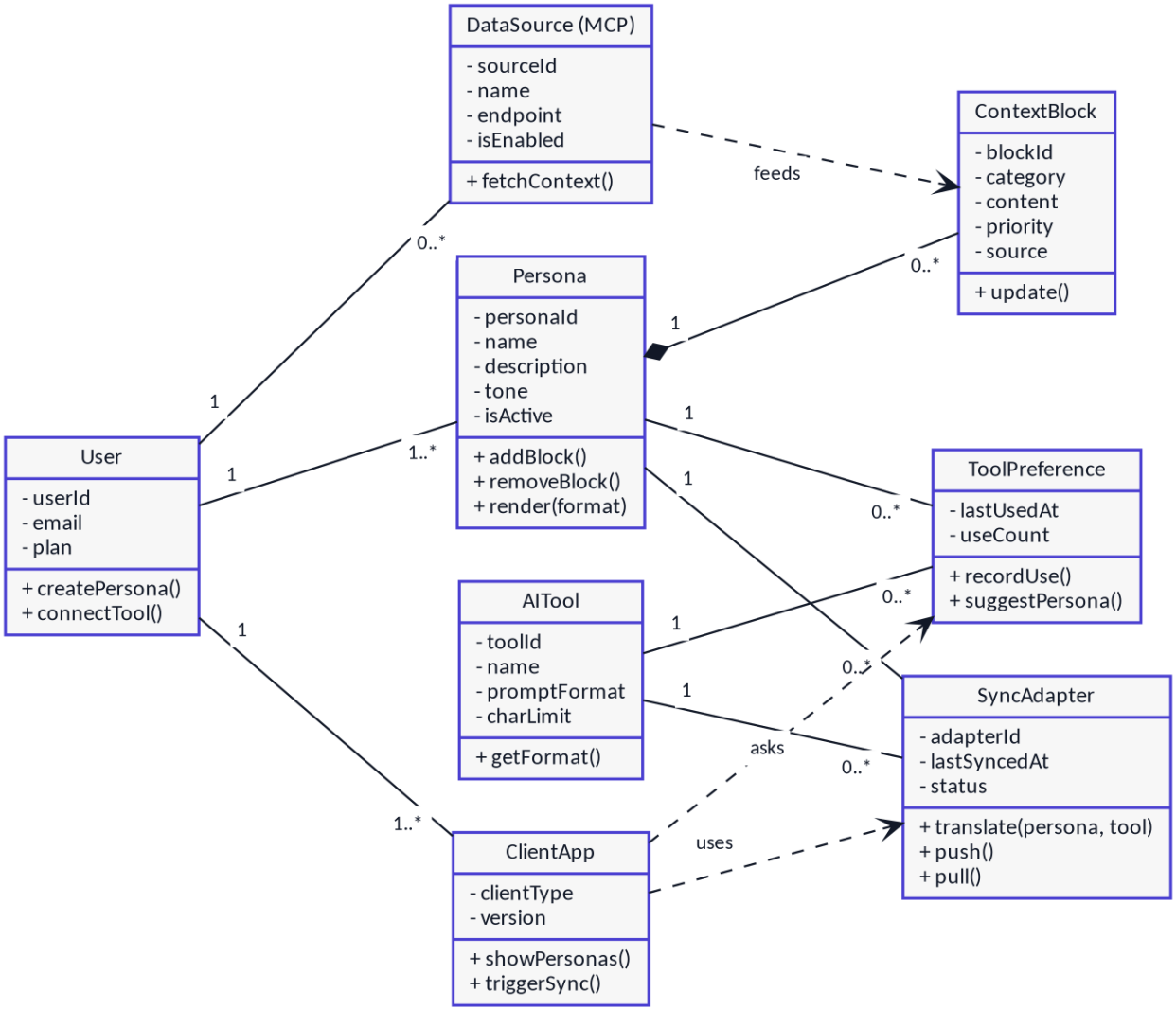


Figure 1. Rough class diagram for PersonaSync™. Solid lines are associations, filled diamond is composition, dashed arrows are dependencies.

Appendix: Templates and Rules

This is the recipe I used for the cards and the diagram, kept here so I can point to it if asked how they were made. The diagram is drawn with Graphviz from a plain-text .dot file, so edits are text edits and the layout is recomputed each time. The PDF is built with a Python script using ReportLab. Both files can live in the course repo next to the SRS if needed.

A.1 CRC card template (simple version)

One card per class. Fill it in with short phrases, not sentences. If the responsibilities column runs past five or six lines, the class is doing too much and should split.

Class: <ClassName>	
Responsibilities	**Collaborators**
• What this class knows (its data) • What this class does (its operations) • One line each, start with a verb	• Other classes it needs to do its job • Only classes it talks to directly

Rules I follow: pick classes from the nouns in the system request. Name each class as a singular noun in UpperCamelCase. A responsibility is something the class knows or does, written as a verb phrase. A collaborator is listed only if this class calls on it directly. Every collaborator named on a card should show up as a line on the class diagram.

A.2 Class diagram template

Each class is a box with three parts: name on top, attributes in the middle, operations on the bottom. Attributes are lowerCamelCase with a minus sign for private. Operations end in parentheses with a plus sign for public.

Lines between boxes and what they mean:

Solid line, no arrow	Association. The two classes know about each other. Put a multiplicity at each end.
Filled diamond	Composition. The part cannot exist without the whole. Diamond sits on the whole.
Dashed line, open arrow	Dependency. One class uses the other but does not own it. Label the line with a verb.
Multiplicity	1 means exactly one. 0..* means zero or more. 1..* means one or more.

Rules I follow: start from the CRC cards, one box per card plus the classes named as collaborators. Lay out the owner at the top or left and the things it owns below or to the right. Keep lines from crossing where possible, since a crossing usually means two classes are placed on the wrong side of each other. Mark the diagram rough if multiplicities or subclasses are still being decided.